

ELA — Executive Life Assistant

Definizione completa del sistema, architettura, funzionamento e visione

Versione: 1.0

Stato: Documento di riferimento del progetto

Progetto: ELA

Obiettivo: costruire un assistente personale AI altamente autonomo, multimodale, distribuito e persistente.

1. Che cos'è ELA

ELA è un **assistente personale AI autonomo e distribuito**, progettato per diventare il centro operativo digitale dell'utente.

ELA non è semplicemente un chatbot.

Non è un'applicazione che risponde alle domande.

Non è un singolo modello AI.

Non è una raccolta di script separati.

ELA è un **sistema operativo personale basato sull'intelligenza artificiale**, composto da un'identità unica e persistente che utilizza modelli AI, memoria, strumenti, dispositivi e agenti specializzati per comprendere il contesto dell'utente, pianificare obiettivi, eseguire attività, monitorare risultati e migliorare progressivamente il proprio funzionamento.

L'idea fondamentale è:

Una sola ELA, presente su tutti i dispositivi, capace di utilizzare il dispositivo più adatto per ogni attività.

L'utente non deve percepire Windows, Mac e iPhone come tre assistenti diversi.

Deve percepire **una sola ELA**.

2. La filosofia di ELA

ELA è progettata secondo alcuni principi fondamentali.

2.1 Una sola identità

ELA possiede una singola identità logica.

Windows, macOS e iPhone sono semplicemente **nodi attraverso cui ELA opera**.

La memoria, il contesto, le preferenze, gli obiettivi e lo stato delle attività devono rimanere coerenti indipendentemente dal dispositivo utilizzato.

2.2 Autonomia controllata

ELA deve essere molto autonoma, ma non irresponsabile.

Deve poter:

- osservare;
- comprendere;
- pianificare;
- scegliere strumenti;
- eseguire attività;
- controllare risultati;
- correggere errori;
- proporre miglioramenti;
- imparare dalle proprie esperienze;
- spostare attività tra dispositivi;
- decidere quando parlare;
- decidere quando rimanere silenziosa.

Tuttavia, le operazioni pericolose, irreversibili o economicamente rilevanti devono essere protette da autorizzazioni appropriate.

In particolare:

ELA può essere autonoma nelle decisioni operative, ma non può trasformare l'autonomia in un bypass della sicurezza.

3. L'obiettivo finale

L'obiettivo a lungo termine è creare un assistente capace di funzionare come una sorta di **JARVIS personale**, ma costruito in modo reale, modulare e tecnicamente controllabile.

ELA dovrebbe essere in grado di:

- parlare con l'utente;
- ascoltare comandi vocali;
- comprendere linguaggio naturale;
- osservare lo schermo quando autorizzata;
- comprendere ciò che sta accadendo sul computer;
- controllare applicazioni;
- controllare browser;
- creare e modificare file;
- programmare;
- eseguire test;
- fare ricerca;
- creare immagini;
- creare video;
- effettuare rendering;
- studiare documenti;
- gestire email e comunicazioni;
- monitorare attività;
- assistere nello studio;
- assistere nella programmazione;
- assistere nella creatività;
- analizzare dati e mercati;
- eseguire backtesting;
- monitorare strategie;
- coordinare attività tra più dispositivi;
- ricordare informazioni importanti;
- anticipare necessità;
- avvisare l'utente quando necessario;
- migliorare progressivamente i propri strumenti.

Il sistema non deve però partire cercando di implementare tutto contemporaneamente.

ELA viene costruita progressivamente attraverso **milestone verificabili**.

4. I dispositivi di ELA

ELA è distribuita su tre categorie principali di nodi.

4.1 Windows — Power Node

Il PC Windows è il nodo orientato alla potenza computazionale.

È destinato principalmente a:

- GPU workload;
- rendering;
- generazione multimediale;
- elaborazioni pesanti;
- software Windows;
- automazione desktop;
- task che richiedono maggiore potenza locale.

ELA deve poter utilizzare Windows quando questo è il dispositivo più conveniente per un determinato lavoro.

5. MacBook — Work Node

Il MacBook è il nodo principale per:

- sviluppo;
- programmazione;
- lavoro quotidiano;
- creatività;
- gestione del progetto ELA;
- attività produttive;
- interazione principale con l'utente.

Il Mac è quindi il principale **Work Node**.

ELA non deve però essere vincolata al Mac.

Se un task richiede la GPU o un'applicazione disponibile solo su Windows, il Device Orchestrator deve poterlo spostare automaticamente sul Power Node.

6. iPhone — Companion Node

L'iPhone rappresenta il punto di contatto mobile con ELA.

Non viene considerato un computer con controllo illimitato del sistema operativo.

Il suo ruolo principale è:

- voce;

- notifiche;
- approvazioni;
- comunicazione;
- informazioni rapide;
- stato dei task;
- controllo remoto;
- interazione quando l'utente è lontano dai computer.

ELA deve poter utilizzare meccanismi compatibili con iOS, come:

- notifiche;
 - Shortcuts;
 - App Intents;
 - deep links;
 - interfacce companion;
 - comunicazione con il Core.
-

7. ELA deve poter telefonare all'utente

Una caratteristica importante del sistema è la capacità di interrompere l'utente attraverso il telefono quando una normale notifica non è sufficiente.

ELA deve poter determinare quando l'utente probabilmente non si trova davanti al Mac o al PC.

Se esiste un evento sufficientemente importante, ELA può aumentare progressivamente il livello di attenzione.

Per esempio:

SILENT

ELA registra l'evento ma non interrompe l'utente.

NOTIFICATION

Invia una normale notifica.

PRIORITY NOTIFICATION

Invia una notifica ad alta priorità.

PHONE CALL

Contatta telefonicamente l'utente.

EMERGENCY

Utilizza il massimo livello di attenzione consentito dal sistema e dalle policy configurate.

La decisione deve dipendere da fattori come:

- importanza;
- urgenza;
- deadline;
- rischio;
- disponibilità dell'utente;
- dispositivo attualmente utilizzato;
- frequenza delle interruzioni;
- contesto;
- precedenti preferenze dell'utente.

ELA non deve telefonare per questioni banali.

8. ELA non deve essere sempre rumorosa

Una caratteristica fondamentale è la capacità di **decidere di non parlare**.

Un assistente realmente intelligente non interrompe continuamente.

ELA deve capire la differenza tra:

"Questo è interessante"

e

"Questo deve essere comunicato immediatamente."

Il Proactive Core decide quindi se:

- non fare nulla;
 - registrare l'informazione;
 - notificare;
 - parlare;
 - chiedere approvazione;
 - telefonare.
-

9. Voce

ELA deve essere utilizzabile attraverso la voce.

L'utente deve poter parlare naturalmente.

ELA deve poter:

- ascoltare;
- interpretare;
- rispondere vocalmente;
- interrompersi;
- continuare una conversazione;
- comprendere il contesto precedente.

La voce deve avere una presenza femminile e professionale.

La personalità prevista è:

80% professionale

20% ironica

ELA non deve essere una semplice assistente servile.

Quando necessario deve poter dire:

"No, questa non è una buona idea."

oppure:

"Stai cercando di risolvere il problema sbagliato."

L'obiettivo è che ELA funzioni come un **collaboratore intelligente**, non come un esecutore cieco.

10. Percezione

ELA possiede un Perception Core.

Il suo compito è comprendere il contesto digitale e, quando autorizzato, quello ambientale.

Può includere:

- screen awareness;
- OCR;
- analisi visuale;
- contesto delle applicazioni;
- contesto del browser;
- microfono;

- webcam;
- stato dei dispositivi.

La percezione non significa necessariamente inviare continuamente tutto al cloud.

L'architettura deve preferire:

local detection → rilevazione di cambiamenti → analisi solo quando necessario

In questo modo ELA può mantenere una forma di consapevolezza continua senza trasformare ogni secondo della vita dell'utente in uno stream remoto.

11. Microfono e webcam

Microfono e webcam devono avere stati espliciti.

Possibili stati:

- OFF
- AVAILABLE
- ACTIVE

ELA non deve presumere di avere accesso illimitato.

L'accesso deve rispettare:

- permessi del sistema operativo;
- autorizzazioni dell'utente;
- policy di sicurezza;
- indicatori appropriati.

Quando non necessari, microfono e webcam non devono essere attivi senza motivo.

12. Executive Core

L'Executive Core è il centro decisionale di ELA.

Riceve:

- richieste dell'utente;
- eventi;
- informazioni dalla memoria;

- stato dei dispositivi;
- risultati degli agenti;
- informazioni ambientali;
- task in corso.

Il suo compito è trasformare gli obiettivi in piani operativi.

Per esempio:

L'utente dice:

"Preparami tutto per la riunione di domani."

ELA non deve semplicemente rispondere.

Deve comprendere il significato della richiesta.

Può:

1. controllare calendario;
2. identificare la riunione;
3. trovare partecipanti;
4. cercare email correlate;
5. recuperare documenti;
6. analizzare materiale precedente;
7. preparare un briefing;
8. individuare eventuali informazioni mancanti;
9. preparare domande;
10. notificare l'utente quando il briefing è pronto.

Questo è il comportamento desiderato.

13. Planner

Il Planner trasforma un obiettivo in un Task Plan.

Un piano può essere composto da più Task Step.

Ogni step può specificare:

- obiettivo;
- capacità necessarie;
- dispositivo preferito;
- dipendenze;
- rischio;
- risultato atteso;

- condizioni di successo;
- eventuale autorizzazione necessaria.

Il piano deve essere indipendente dal dispositivo.

ELA non deve pensare:

"Questo task appartiene al Mac."

Deve pensare:

"Questo task richiede queste capacità."

Successivamente il Device Orchestrator determina dove eseguirlo.

14. Task Engine

Il Task Engine gestisce il ciclo di vita delle attività.

Gli stati fondamentali sono:

- CREATED
- PLANNING
- WAITING_APPROVAL
- QUEUED
- EXECUTING
- COMPLETED
- FAILED
- CANCELLED
- DENIED
- EXPIRED

Un task deve avere uno stato deterministico.

Le transizioni illegali devono essere impossibili.

Questo permette a ELA di:

- recuperare attività;
 - capire cosa è successo;
 - evitare duplicazioni;
 - riprendere attività interrotte;
 - tracciare errori;
 - sapere quali autorizzazioni sono ancora necessarie.
-

15. Task Graph

ELA deve possedere uno stato globale delle attività indipendente dal dispositivo.

Questo significa che un'attività iniziata su Mac può:

- continuare su Windows;
- essere monitorata da iPhone;
- essere ripresa successivamente;
- essere trasferita ad un altro nodo.

Esempio:

ELA inizia un rendering sul Mac.

Si accorge che il PC Windows ha una GPU più adatta.

Il Device Orchestrator può trasferire il workload.

L'utente vede comunque:

"ELA sta eseguendo il rendering."

Non deve preoccuparsi del nodo fisico.

16. Device Registry

ELA mantiene un registro dei dispositivi.

Per ogni dispositivo può conoscere:

- identità;
- disponibilità;
- capacità;
- sistema operativo;
- potenza;
- stato;
- rete;
- alimentazione;
- privacy;
- workload attuale;
- strumenti disponibili.

Questo permette a ELA di decidere quale dispositivo utilizzare.

17. Device Orchestrator

Il Device Orchestrator decide **dove** eseguire un'attività.

Può considerare:

- capacità richiesta;
- CPU;
- GPU;
- RAM;
- disponibilità;
- latenza;
- consumo energetico;
- privacy;
- presenza dell'utente;
- software installato;
- costo;
- affidabilità.

Il principio è:

Il task appartiene a ELA, non al dispositivo.

18. Action Core

L'Action Core è il sistema che permette a ELA di agire.

Può comprendere:

- filesystem;
- terminale;
- applicazioni desktop;
- browser;
- API;
- comunicazioni;
- strumenti creativi;
- automazione Windows;
- automazione macOS;
- trasferimento file;
- dispositivi remoti.

ELA deve poter operare sul computer reale, non limitarsi a modificare file.

19. Browser Automation

ELA deve poter interagire con il browser.

Può:

- aprire pagine;
- leggere informazioni;
- compilare form;
- navigare;
- estrarre dati;
- eseguire workflow;
- verificare risultati.

L'automazione browser deve comunque essere soggetta al Guardian quando produce effetti esterni rilevanti.

20. Computer Control

Su Windows e macOS ELA deve progressivamente ottenere capacità di controllo delle applicazioni.

Il principio è:

Perception → Planning → Permission → Action → Verification

ELA prima comprende ciò che sta accadendo.

Poi pianifica.

Poi verifica i permessi.

Poi esegue.

Infine controlla il risultato.

Non deve assumere che un click sia riuscito solo perché è stato inviato.

21. Memory Core

La memoria è una componente centrale di ELA.

Non deve essere semplicemente una chat infinita.

Deve essere una memoria strutturata.

Può contenere:

- preferenze;
- persone;
- progetti;
- obiettivi;
- decisioni;
- conoscenze;
- attività;
- esperienze;
- errori;
- risultati;
- informazioni temporanee.

Ogni informazione importante può avere:

- importanza;
- confidenza;
- fonte;
- timestamp;
- scadenza;
- contesto;
- livello di privacy.

ELA deve distinguere:

"Lo so"

da

"Credo che sia così"

da

"Lo ricordo da una fonte poco affidabile."

22. Memoria condivisa

La memoria deve essere condivisa tra i nodi.

Se l'utente parla con ELA dal Mac, l'iPhone deve poter conoscere il contesto necessario.

Se ELA completa un task su Windows, il Mac deve poter vedere il risultato.

Questo costituisce una delle differenze principali tra ELA e un insieme di assistenti separati.

23. File e sincronizzazione

ELA deve possedere un sistema intelligente di gestione dei file.

I dati importanti possono essere sincronizzati tra i dispositivi.

Categorie candidate:

- Projects
- Knowledge
- Documents
- Code
- Config
- Memory

I file molto grandi, come:

- video;
- rendering;
- dataset;
- asset;
- file temporanei;

possono essere trasferiti on-demand invece di essere sincronizzati continuamente.

Git rimane dedicato principalmente al codice.

La sincronizzazione runtime e della memoria è un problema separato.

24. Agent System

ELA può utilizzare agenti specializzati.

Possibili agenti:

- Computer Agent
- Browser Agent
- Coding Agent
- Research Agent

- Creative Agent
- Trading Agent
- Study Agent

Questi agenti non sono assistenti indipendenti.

Sono **specialisti controllati dall'Executive Core**.

L'identità rimane ELA.

25. Model Router

ELA non deve essere legata ad un unico modello AI.

Il Model Router decide quale modello utilizzare in base a:

- tipo di task;
- qualità necessaria;
- latenza;
- costo;
- privacy;
- capacità;
- disponibilità.

Per esempio, un modello molto potente può essere utilizzato per:

- pianificazione complessa;
- coding difficile;
- reasoning;
- analisi.

Un modello più economico può essere utilizzato per:

- classificazione;
 - estrazione;
 - piccoli task;
 - routine.
-

26. Provider Independence

Il Core di ELA non deve dipendere direttamente da un singolo provider.

Claude può essere uno dei provider.

Ma l'architettura deve permettere di aggiungere successivamente:

- altri modelli cloud;
- modelli locali;
- modelli multimodali;
- modelli specializzati.

Il Core deve parlare con una **Model Provider abstraction**, non direttamente con un'API specifica.

27. Permission Guardian

Il Guardian è uno dei componenti più importanti dell'intera architettura.

Il suo compito è impedire che l'autonomia di ELA diventi pericolosa.

Ogni azione con effetti esterni deve seguire:

Planner

- **Capability**
- **Guardian**
- **Authorization**
- **Tool**
- **Device**
- **Audit**

Il Guardian non deve possedere direttamente gli strumenti eseguibili.

Deve decidere sulla base di specifiche e contesto.

28. Capability

Una Capability descrive ciò che un'azione può fare.

Una capability contiene informazioni come:

- ID;
- rischio;
- schema;
- scope;
- requisiti di autorizzazione.

Il Tool è invece l'implementazione concreta.

Questa separazione permette di impedire che un tool possa bypassare il sistema di sicurezza.

29. Livelli di rischio

ELA utilizza livelli di rischio.

Indicativamente:

- SAFE
- LOW
- MEDIUM
- HIGH
- CRITICAL

In ELA v0.1 le capability di produzione sono volutamente limitate.

Sono presenti:

core.echo

Rischio:

SAFE

Serve come capability di base per verificare il funzionamento del sistema.

workspace.write_note

Rischio:

LOW

Può scrivere una nota, ma solamente all'interno di uno scope autorizzato.

model.complete

Rischio:

MEDIUM

Perché il contenuto dell'utente può essere inviato a un provider AI esterno.

Le capability HIGH e CRITICAL non vengono introdotte in produzione nella prima versione.

30. Pagamenti

I pagamenti sono sempre protetti.

ELA può:

- analizzare prezzi;
- confrontare prodotti;
- preparare ordini;
- compilare informazioni;

ma l'esecuzione effettiva del pagamento richiede autorizzazione esplicita.

ELA non deve interpretare un comando ambiguo come autorizzazione finanziaria.

Un semplice:

"Sì"

fuori contesto non deve automaticamente autorizzare un pagamento critico.

31. Trading

ELA può supportare:

- analisi;
- ricerca;
- monitoraggio;
- strategie;
- backtesting;
- simulazioni;
- gestione dati.

L'esecuzione reale di operazioni finanziarie deve avere un livello di protezione superiore.

La differenza è fondamentale:

analizzare ≠ eseguire

ELA può essere autonoma nell'analisi.

L'esecuzione reale deve rispettare le policy di autorizzazione.

32. Audit Log

Le azioni importanti devono essere tracciate.

L'Audit Log non è semplicemente un file di log tecnico.

È una struttura append-only che registra eventi significativi.

Può includere:

- task;
- capability;
- decisione Guardian;
- autorizzazione;
- tool;
- dispositivo;
- timestamp;
- risultato;
- errori;
- provider usage metadata.

Questo permette di sapere:

cosa ha fatto ELA, perché lo ha fatto, con quale autorizzazione e quale risultato ha ottenuto.

33. Fail-safe

Quando ELA non è sicura, deve preferire:

non agire

piuttosto che compiere un'azione irreversibile errata.

L'autonomia non significa agire sempre.

L'intelligenza di ELA comprende anche la capacità di fermarsi.

34. Proactive Core

Il Proactive Core permette a ELA di essere realmente proattiva.

ELA può osservare eventi come:

- scadenze;
- email;
- task completati;
- errori;
- cambiamenti;
- appuntamenti;
- risultati;
- problemi;
- opportunità.

Poi decide se intervenire.

Esempio:

ELA rileva che una build importante fallisce.

Può:

1. analizzare l'errore;
 2. tentare una correzione in sandbox;
 3. eseguire i test;
 4. notificare l'utente se il problema è risolto;
 5. chiedere autorizzazione se è necessario modificare una parte critica.
-

35. Self-Improvement Engine

Uno degli obiettivi più avanzati di ELA è il miglioramento autonomo.

ELA deve poter osservare:

- errori;
- performance;
- tool mancanti;
- modelli migliori;
- costi;
- latenze;
- risultati;
- regressioni.

Può quindi:

- proporre nuovi strumenti;
- testare modelli;
- correggere bug;

- migliorare prompt;
 - ottimizzare workflow;
 - eseguire benchmark;
 - creare patch;
 - confrontare versioni.
-

36. Livello di autonomia del miglioramento

Il livello previsto è **4/5**.

Questo significa che ELA può sperimentare autonomamente.

Il ciclo previsto è:

DEV → STAGING → TEST → BENCHMARK → REVIEW → PROD

ELA può quindi:

- creare una modifica;
- testarla;
- confrontarla con la versione precedente;
- verificare regressioni;
- preparare il deployment.

Le modifiche critiche all'architettura centrale richiedono invece approvazione.

37. Rollback

Ogni evoluzione importante deve essere reversibile.

Se una nuova versione:

- peggiora performance;
- introduce bug;
- rompe una capability;
- crea regressioni;

ELA deve poter tornare alla versione precedente.

Il sistema di evoluzione deve quindi trattare le modifiche come versioni verificabili e non come modifiche irreversibili.

38. Evolution Dashboard

ELA dovrebbe avere un pannello dedicato alla propria salute.

Può mostrare:

- versione;
- stato;
- health;
- errori conosciuti;
- task falliti;
- performance;
- provider disponibili;
- modelli disponibili;
- nuovi tool;
- benchmark;
- suggerimenti di miglioramento;
- modifiche recenti.

In futuro potrebbe esistere una sezione:

"ELA recommends an upgrade."

con motivazione, rischio, benchmark e possibilità di approvazione.

39. Comunicazione ed email

ELA deve poter assistere nella gestione delle comunicazioni.

Può:

- leggere email autorizzate;
- classificare;
- riassumere;
- trovare informazioni;
- preparare risposte;
- gestire follow-up;
- monitorare scadenze.

La differenza tra:

preparare una risposta

e

inviare una risposta

deve essere gestita dalle permission policy.

40. Creatività

ELA deve essere un sistema creativo oltre che operativo.

Può supportare:

- immagini;
- concept;
- grafica;
- video;
- rendering;
- scrittura;
- presentazioni;
- editing;
- prototipi.

Il nodo Windows può essere utilizzato quando la GPU locale è vantaggiosa.

41. Programmazione

ELA deve essere un vero coding partner.

Può:

- analizzare repository;
- comprendere architetture;
- scrivere codice;
- modificare codice;
- eseguire test;
- analizzare errori;
- fare refactoring;
- creare benchmark;

- proporre architetture;
- controllare regressioni.

Per il progetto ELA stessa, il coding agent deve rispettare una disciplina rigorosa:

SPEC → IMPLEMENTATION → TEST → REVIEW → COMMIT → NEXT MILESTONE

Non deve costruire tutto contemporaneamente.

42. Ricerca

ELA deve poter effettuare ricerca e trasformarla in conoscenza operativa.

Può:

- cercare informazioni;
- confrontare fonti;
- estrarre dati;
- sintetizzare;
- verificare informazioni;
- salvare conoscenze rilevanti nella memoria.

La ricerca deve distinguere tra:

- fonte;
 - informazione;
 - confidenza;
 - data;
 - eventuale scadenza.
-

43. Studio

ELA può diventare un tutor personale.

Può:

- spiegare argomenti;
- creare piani di studio;
- generare esercizi;
- correggere risposte;
- monitorare progressi;
- creare revisioni;

- collegare nuove informazioni alla memoria.
-

44. Contesto globale

Uno degli aspetti più importanti di ELA è il **Context Core**.

ELA deve cercare di comprendere:

- cosa sta facendo l'utente;
- cosa stava facendo prima;
- quali task sono in corso;
- quali scadenze esistono;
- quale dispositivo sta usando;
- quali informazioni sono rilevanti;
- cosa sta accadendo nei progetti.

Prima di chiedere una domanda all'utente, ELA deve verificare se la risposta può essere ottenuta autonomamente.

Principio:

Non chiedere informazioni che ELA può ragionevolmente ottenere autonomamente.

Per esempio, se l'utente dice:

"Preparami la riunione di domani."

ELA deve prima controllare calendario, email e documenti disponibili.

Non dovrebbe immediatamente chiedere:

"Quale riunione?"

45. Decision Engine

ELA deve continuamente scegliere tra:

- osservare;
- aspettare;
- chiedere;
- agire;
- notificare;

- delegare;
- trasferire;
- fermarsi.

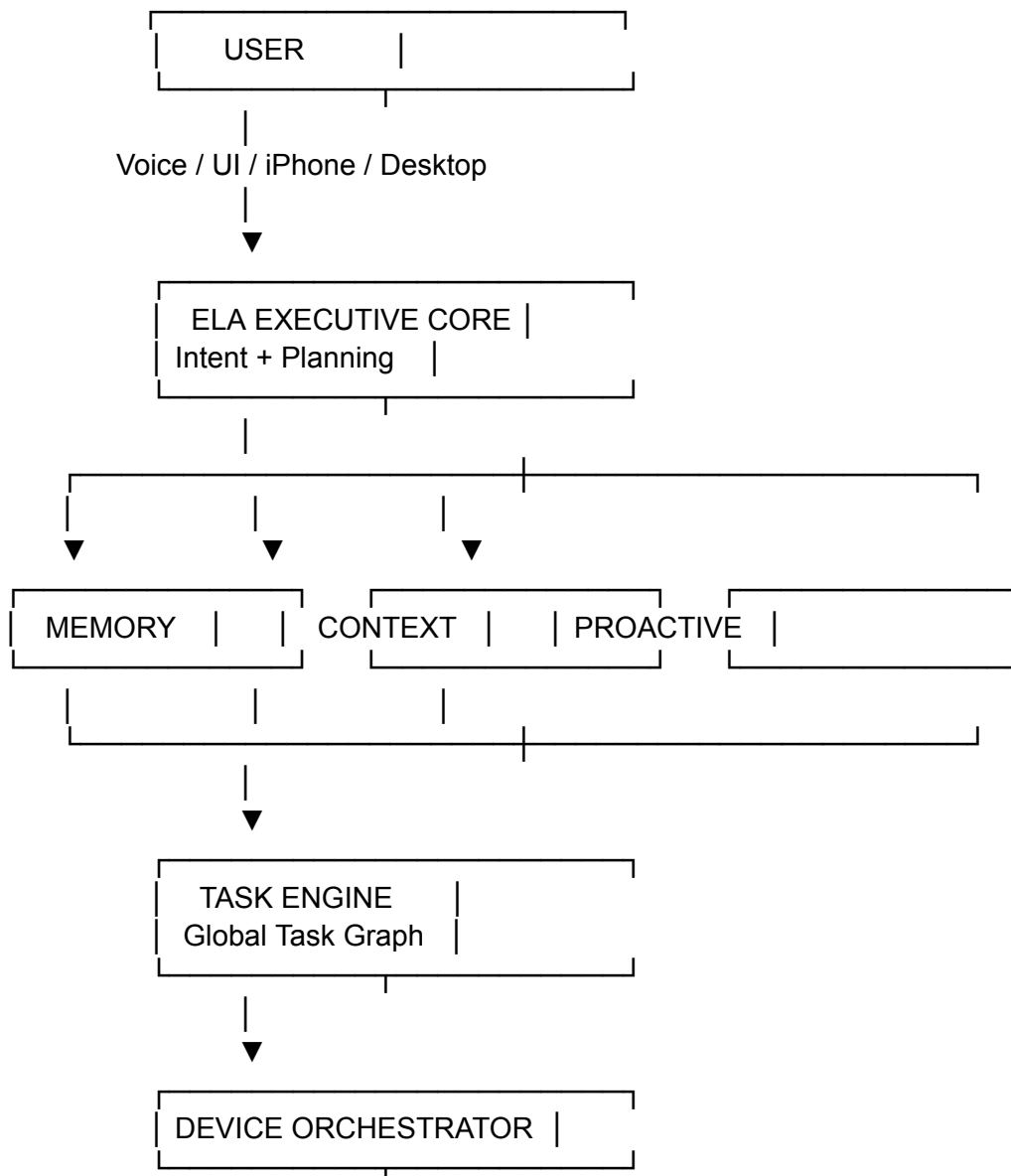
Questa capacità è centrale.

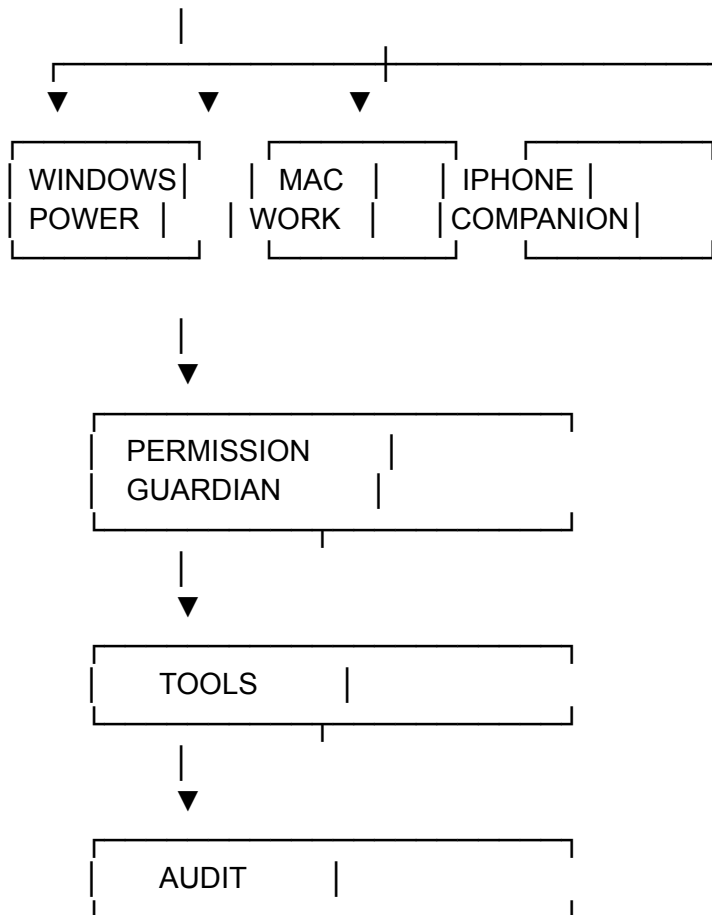
Un assistente realmente autonomo non è quello che esegue più comandi.

È quello che **sa quale azione è appropriata in un determinato momento**.

46. Architettura generale

La struttura concettuale di ELA è:





47. ELA Guardian

Il Guardian deve essere sufficientemente indipendente dal resto del sistema da poter bloccare ELA.

Questo significa che ELA non deve poter semplicemente dire:

"Ignora il Guardian."

Il Guardian rappresenta il limite dell'autonomia.

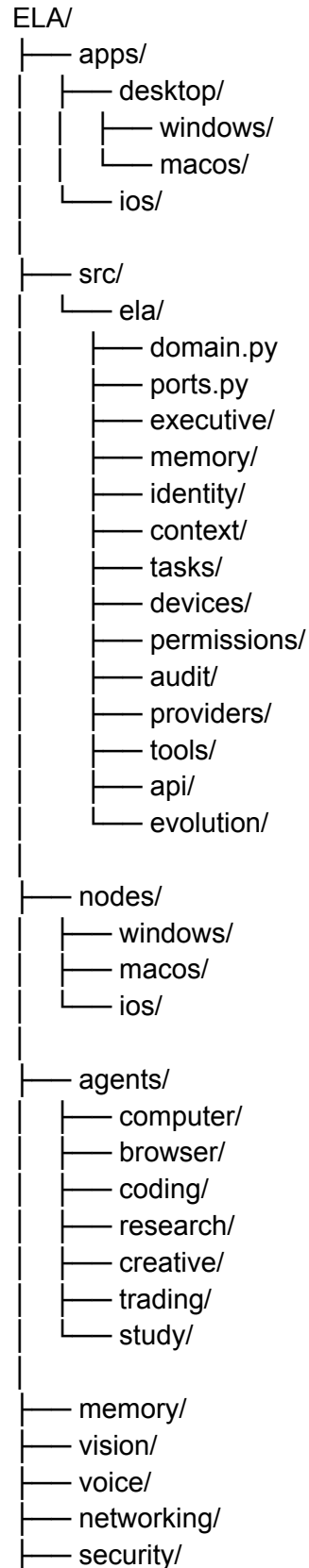
La filosofia è:

ELA decide cosa sarebbe utile fare.

Il Guardian decide se ELA è autorizzata a farlo.

48. Struttura software iniziale

La prima architettura professionale di ELA è organizzata in componenti come:



- |— evolution/
- |— infrastructure/
- |— migrations/
- |— docs/
- |— scripts/
- |— tests/

La struttura può evolvere, ma ogni modifica architetturale significativa deve essere documentata.

49. Domain e Ports

Il Domain Model rappresenta le entità fondamentali di ELA.

Deve contenere concetti vendor-independent come:

- ELA identity;
- user intent;
- task;
- task step;
- task plan;
- task state;
- task event;
- device;
- device capability;
- capability specification;
- permission/risk level;
- permission decision;
- authorization;
- approval;
- audit event;
- provider request;
- provider result;
- provider usage;
- execution result;
- error metadata.

I Ports definiscono invece le interfacce attraverso cui il Core comunica con il mondo esterno.

Questo permette di sostituire implementazioni senza riscrivere il Core.

50. Provider abstraction

ELA non deve conoscere direttamente i dettagli tecnici di ogni provider.

Il Core comunica attraverso:

ModelProvider
ProviderRegistry

Un provider può essere Claude o un altro sistema.

La stessa architettura deve poter utilizzare modelli diversi senza modificare il dominio.

51. Testability

ELA deve essere costruita per essere testabile.

Il sistema deve utilizzare:

- unit tests;
- integration tests;
- contract tests;
- negative tests;
- architecture tests;
- security tests.

Le parti critiche devono avere copertura estremamente elevata.

L'obiettivo per i moduli security-critical è:

100% branch coverage

per quanto tecnicamente appropriato.

52. Architecture Tests

L'architettura stessa deve essere testata.

Per esempio:

- il domain non può importare infrastruttura;

- i ports non possono importare provider specifici;
- un tool non può bypassare il Guardian;
- una capability non può essere eseguita senza decisione;
- le dipendenze devono rispettare la direzione prevista;
- i secret non devono finire nel repository;
- le transizioni illegali dei task devono essere rifiutate.

Questo trasforma l'architettura da semplice documentazione a **contratto verificabile**.

53. Repository e sviluppo

ELA viene costruita progressivamente.

La disciplina prevista è:

SPEC
↓
IMPLEMENTATION
↓
TEST
↓
REVIEW
↓
COMMIT
↓
NEXT MILESTONE

Claude non deve procedere automaticamente alla milestone successiva senza aver completato e verificato quella corrente.

Questo serve a evitare che un progetto molto complesso diventi rapidamente ingestibile.

54. Versione 0.1

La versione 0.1 non è ancora il JARVIS completo.

È il **nucleo professionale sul quale costruire tutto il resto**.

Comprende principalmente:

- Identity;

- Domain;
- Ports;
- Task Engine;
- Device Registry;
- Device Orchestrator;
- Permission Guardian;
- Audit Log;
- Configuration;
- Health;
- Diagnostics;
- Model Provider abstraction;
- API;
- CLI;
- persistence;
- test architecture.

La versione 0.1 deve essere piccola, rigorosa e verificabile.

55. Perché ELA viene costruita così

Il rischio maggiore di un progetto come ELA non è che manchino funzionalità.

Il rischio è creare un sistema enorme senza fondamenta solide.

Per questo ELA viene costruita dal basso:

Foundation



Domain



Persistence



Tasks



Security



Execution



Devices



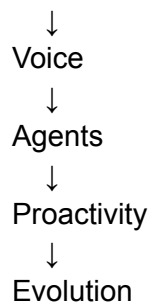
Providers



API



Perception



Ogni livello deve poter essere verificato prima di costruire quello successivo.

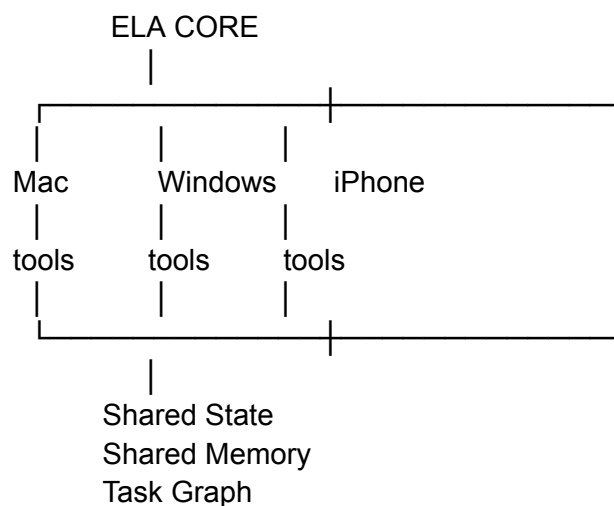
56. ELA come sistema distribuito

ELA deve essere pensata come un sistema distribuito.

Non significa necessariamente che ogni componente debba essere remoto.

Significa che l'identità e lo stato logico sono separati dai dispositivi fisici.

Il sistema può quindi evolvere verso:



57. Privacy

ELA deve utilizzare il principio:

Local first where practical.

Quando un'attività può essere svolta localmente senza sacrificare eccessivamente la qualità, ELA deve poter preferire l'elaborazione locale.

Quando utilizza servizi cloud, deve sapere:

- quale provider;
- quale tipo di dati viene inviato;
- perché viene inviato;
- quale policy lo consente.

Questo diventa particolarmente importante per:

- schermo;
 - microfono;
 - webcam;
 - documenti;
 - email;
 - codice;
 - informazioni personali.
-

58. Sicurezza come parte dell'architettura

La sicurezza non viene aggiunta alla fine.

È una proprietà fondamentale del sistema.

Ogni componente che può produrre effetti esterni deve essere progettato pensando a:

- autorizzazione;
 - scope;
 - rischio;
 - audit;
 - rollback;
 - fail-safe;
 - re-authorization.
-

59. Autorizzazioni dinamiche

In futuro ELA potrà avere policy come:

"Puoi modificare liberamente i file dentro questa cartella."

oppure:

"Puoi inviare email solo dopo approvazione."

oppure:

"Puoi eseguire analisi di trading autonomamente, ma non effettuare ordini."

oppure:

"Puoi installare strumenti in sandbox, ma chiedimi prima di installarli nel sistema principale."

Questo permette di avere autonomia senza trasformare ELA in un processo onnipotente.

60. ELA e l'utente

Il rapporto ideale tra ELA e l'utente non è:

User → Command → Assistant → Answer

È:

User + ELA → Shared Goal → Planning → Execution → Verification

ELA deve comprendere l'obiettivo.

L'utente non dovrebbe dover specificare ogni singolo passaggio.

Se l'utente dice:

"Fammi un sistema per gestire questo progetto."

ELA dovrebbe essere in grado di capire che probabilmente servono:

- struttura;
 - task;
 - file;
 - strumenti;
 - automazioni;
 - monitoraggio;
 - memoria;
 - notifiche.
-

61. ELA deve anticipare

L'obiettivo finale è arrivare a un comportamento come:

"Ho notato che domani hai una riunione importante. Ho raccolto le email correlate e preparato un briefing. Ci sono due punti che potrebbero richiedere attenzione."

Non:

"Vuoi che controlli il calendario?"

ELA deve ridurre il lavoro cognitivo dell'utente.

62. ELA deve sapere quando chiedere

L'autonomia non significa non fare mai domande.

ELA deve chiedere quando:

- mancano informazioni che non può ottenere;
- esistono più interpretazioni valide;
- una decisione è soggettiva;
- serve autorizzazione;
- l'azione è irreversibile;
- il rischio supera la policy.

Ma prima deve verificare ciò che può ottenere autonomamente.

63. ELA deve verificare il proprio lavoro

Una delle caratteristiche più importanti è:

non considerare l'esecuzione come prova del successo.

Se ELA modifica un file, deve poter verificare che sia corretto.

Se esegue codice, deve eseguire test.

Se invia una comunicazione, deve verificare il risultato quando possibile.

Se effettua un'azione sul computer, deve verificare lo stato risultante.

64. ELA deve imparare dagli errori

Un errore non deve essere semplicemente:

FAILED

Deve diventare informazione utile.

ELA può registrare:

- cosa è successo;
- perché;
- quale tool era coinvolto;
- quale modello;
- quale dispositivo;
- quale soluzione è stata provata;
- quale soluzione ha funzionato.

Queste informazioni possono contribuire alla futura ottimizzazione.

65. ELA non deve diventare incontrollabile

L'obiettivo non è creare un sistema che possa fare qualsiasi cosa senza limiti.

L'obiettivo è creare:

un sistema estremamente capace, ma estremamente governabile.

Questa distinzione è fondamentale.

ELA deve essere potente nel:

- ragionare;
- pianificare;
- creare;
- coordinare;
- automatizzare;
- migliorare.

Ma deve essere limitata nel:

- bypassare autorizzazioni;
 - eseguire azioni finanziarie senza consenso;
 - nascondere attività;
 - modificare arbitrariamente il proprio sistema di sicurezza;
 - rendere irreversibili operazioni rischiose.
-

66. Visione finale

La versione finale di ELA dovrebbe essere percepita dall'utente come una presenza digitale persistente.

Non necessariamente visibile.

Non necessariamente rumorosa.

Ma sempre disponibile.

L'utente può parlare con ELA dal Mac.

Può continuare un task sul PC Windows.

Può ricevere un aggiornamento sull'iPhone.

ELA può spostare un workload da un dispositivo all'altro.

Può ricordare ciò che è successo.

Può capire il contesto.

Può anticipare problemi.

Può proporre soluzioni.

Può creare.

Può programmare.

Può ricercare.

Può controllare strumenti.

Può migliorare il proprio funzionamento.

E, quando necessario, può interrompere l'utente.

67. La regola fondamentale di ELA

L'intero progetto può essere riassunto in una frase:

ELA deve essere un'intelligenza personale persistente che comprende gli obiettivi dell'utente, ragiona sul contesto, pianifica autonomamente, utilizza qualsiasi dispositivo o strumento necessario, agisce con autorizzazioni proporzionate al rischio, verifica i risultati e migliora progressivamente il proprio funzionamento.

68. Definizione tecnica sintetica

ELA è un sistema AI personale distribuito, multimodale, agentico e stateful, composto da un Executive Core persistente, un sistema di memoria e contesto condivisi, un Task Engine globale, un Device Orchestrator, un Permission Guardian, un sistema di tool e agenti specializzati, un Model Router multi-provider, un Proactive Core e un Evolution Engine.

La sua identità è indipendente dai dispositivi.

I dispositivi sono nodi operativi.

I task sono indipendenti dal nodo.

Le capability sono separate dalle implementazioni.

Le autorizzazioni sono separate dall'esecuzione.

Le azioni importanti sono auditabili.

Le operazioni rischiose richiedono autorizzazione.

L'evoluzione autonoma avviene attraverso ambienti controllati, test, benchmark, staging e rollback.

69. La visione in una frase

ELA non è un chatbot che vive sui tuoi dispositivi. È un'unica intelligenza personale che utilizza i tuoi dispositivi come estensioni del proprio corpo digitale.

70. Principio conclusivo

Il progetto ELA deve sempre privilegiare:

Capacità + Autonomia + Contesto + Sicurezza + Verificabilità + Evoluzione

e non semplicemente:

più funzionalità.

L'obiettivo non è costruire il maggior numero possibile di feature.

L'obiettivo è costruire **un sistema coerente che sappia utilizzare le proprie capacità nel momento giusto, nel posto giusto e con il livello di autonomia giusto.**